

Name: Hồ Huyền Trang

Student ID: 11247236

Class: DS66B

Lab Exercise 1: Triggering Nested Loop Join

The screenshot shows a PostgreSQL query editor with a query and its execution plan.

Query:

```
1 EXPLAIN (ANALYZE, BUFFERS)
2 SELECT c.first_name, c.last_name, r.rental_date
3 FROM customer c
4 JOIN rental r
5     ON c.customer_id = r.customer_id
6 WHERE c.customer_id = 1;
```

Execution Plan:

Step	Operation	Cost	Rows	Width	Actual Time	Actual Rows	Actual Loops
1	Nested Loop	0.28..359.16	32	21	1.318..4.793	32.00	1
2	Buffers: shared read=153 dirtied=1						
3	-> Index Scan using customer_pkey on customer c	0.28..8.29	1	17	0.696..0.698	1.00	1
4	Index Cond: (customer_id = 1)						
5	Index Searches: 1						
6	Buffers: shared read=3						
7	-> Seq Scan on rental r	0.00..350.55	32	10	0.617..4.073	32.00	1
8	Filter: (customer_id = 1)						
9	Rows Removed by Filter: 16012						
10	Buffers: shared read=150 dirtied=1						
11	Planning:						
12	Buffers: shared hit=122 read=6 dirtied=1						
13	Planning Time: 4.289 ms						
14	Execution Time: 4.898 ms						

Total rows: 14 | Query complete 00:00:00.047

Question:

1. Which table is the Outer (top) and which is the Inner (bottom)?

The customer table is the Outer table, while the rental table is the Inner table. The execution plan shows an Index Scan on customer followed by a Sequential Scan on rental.

2. Why Nested Loop? How many rows did the customer table return? Why does this make Hash Join "too expensive"?

The customer table returned only 1 row because of the condition `customer_id = 1`. This makes Nested Loop suitable because the outer table is very small. A Hash Join would introduce additional overhead for building and probing a hash table, which is unnecessary for such a small result.

3. Look at loops in the Inner Table node. Why is it 1?

The Inner table has loops = 1 because the Outer table (customer) returned only 1 row. Therefore, PostgreSQL executed the Inner scan only once. The Inner scan returned 32 rental records matching `customer_id = 1`.

Lab Exercise 2: Triggering Hash Join

```

8  EXPLAIN (ANALYZE, BUFFERS)
9  SELECT f.title, l.name AS language
10 FROM film f
11 JOIN language l
12     ON f.language_id = l.language_id;

```

Data Output Messages Notifications

Showing rows: 1 to 14 Page No: 1 of 1

	QUERY PLAN
1	Hash Join (cost=1.14..103.62 rows=1000 width=99) (actual time=0.351..3.288 rows=1000.00 loops=1)
2	Hash Cond: (f.language_id = l.language_id)
3	Buffers: shared hit=39 read=50
4	-> Seq Scan on film f (cost=0.00..98.00 rows=1000 width=17) (actual time=0.017..2.704 rows=1000.00 loops=1)
5	Buffers: shared hit=39 read=49
6	-> Hash (cost=1.06..1.06 rows=6 width=88) (actual time=0.312..0.313 rows=6.00 loops=1)
7	Buckets: 1024 Batches: 1 Memory Usage: 9kB
8	Buffers: shared read=1
9	-> Seq Scan on language l (cost=0.00..1.06 rows=6 width=88) (actual time=0.278..0.279 rows=6.00 loops=1)
10	Buffers: shared read=1
11	Planning:
12	Buffers: shared hit=175 read=2
13	Planning Time: 1.721 ms
14	Execution Time: 3.363 ms

Total rows: 14 Query complete 00:00:00.042 LF

```

14 SET work_mem = '64kB';

```

Data Output Messages Notifications

SET

Query returned successfully in 40 msec.

```

8  EXPLAIN (ANALYZE, BUFFERS)
9  SELECT f.title, l.name AS language
10 FROM film f
11 JOIN language l
12     ON f.language_id = l.language_id;
13
14 SET work_mem = '64kB';

```

Data Output Messages Notifications

Showing rows: 1 to 12 Page No: 1 of 1

	QUERY PLAN
1	Hash Join (cost=1.14..103.62 rows=1000 width=99) (actual time=0.104..0.835 rows=1000.00 loops=1)
2	Hash Cond: (f.language_id = l.language_id)
3	Buffers: shared hit=89
4	-> Seq Scan on film f (cost=0.00..98.00 rows=1000 width=17) (actual time=0.057..0.446 rows=1000.00 loops=1)
5	Buffers: shared hit=88
6	-> Hash (cost=1.06..1.06 rows=6 width=88) (actual time=0.035..0.036 rows=6.00 loops=1)
7	Buckets: 1024 Batches: 1 Memory Usage: 9kB
8	Buffers: shared hit=1
9	-> Seq Scan on language l (cost=0.00..1.06 rows=6 width=88) (actual time=0.016..0.018 rows=6.00 loops=1)
10	Buffers: shared hit=1
11	Planning Time: 0.389 ms
12	Execution Time: 0.960 ms

Total rows: 12 Query complete 00:00:00.043 L

Question:

4. Which table is being Hashed (Built)? Which table is being Probed?

The language table is the hashed table because PostgreSQL builds a hash table from its 6 rows. The film table is the probe side because PostgreSQL scans its 1,000 rows and checks their language_id against the hash table.

5. Look at the Hash node. What is the Memory Usage?

The Hash node uses **9kB of memory**. The execution plan also shows **Batches = 1**, meaning that the hash table fits entirely in memory.

6. What happens if you run `SET work_mem = '64kB'`; and run it again? (Try it!)

After setting `work_mem` to 64kB, the execution plan remained a Hash Join. The Hash node still had **Batches = 1** and **Memory Usage = 9kB**. This is because the hash table requires only 9kB, which is smaller than the 64kB `work_mem` limit. Therefore, PostgreSQL can keep the entire hash table in memory without spilling to disk.

Lab Exercise 3: Triggering Merge Join

```
16 SET enable_hashjoin = OFF;
17 SET enable_nestloop = OFF;
18
19 EXPLAIN (ANALYZE, BUFFERS)
20 SELECT f.film_id, f.title, i.inventory_id
21 FROM film f
22 JOIN inventory i
23     ON f.film_id = i.film_id
24 ORDER BY f.film_id;
```

Data Output Messages Notifications

Showing rows: 1 to 19 Page No: 1 of 1

	QUERY PLAN	
	text	
1	Merge Join (cost=412.64..622.36 rows=4581 width=23) (actual time=3.513..5.662 rows=4581.00 loops=1)	
2	Merge Cond: (f.film_id = i.film_id)	
3	Buffers: shared hit=86 read=26, temp read=10 written=12	
4	-> Index Scan using film_pkey on film f (cost=0.28..127.33 rows=1000 width=19) (actual time=0.014..0.514 rows=1000.00 loops=1)	
5	Index Searches: 1	
6	Buffers: shared hit=83 read=1	
7	-> Materialize (cost=412.37..435.27 rows=4581 width=6) (actual time=3.486..4.336 rows=4581.00 loops=1)	
8	Storage: Memory Maximum Storage: 17kB	
9	Buffers: shared hit=3 read=25, temp read=10 written=12	
10	-> Sort (cost=412.37..423.82 rows=4581 width=6) (actual time=3.470..3.812 rows=4581.00 loops=1)	
11	Sort Key: i.film_id	
12	Sort Method: external merge Disk: 80kB	
13	Buffers: shared hit=3 read=25, temp read=10 written=12	
14	-> Seq Scan on inventory i (cost=0.00..70.81 rows=4581 width=6) (actual time=0.652..1.689 rows=4581.00 loops=1)	
Total rows: 19 Query complete 00:00:00.032		LF

19	EXPLAIN (ANALYZE, BUFFERS)
20	SELECT f.film_id, f.title, i.inventory_id
21	FROM film f
22	JOIN inventory i
23	ON f.film_id = i.film_id
24	ORDER BY f.film_id;

Data Output	Messages	Notifications
Showing rows: 1 to 19		
Page No: 1 of 1		
6	Buffers: shared hit=83 read=1	
7	-> Materialize (cost=412.37..435.27 rows=4581 width=6) (actual time=3.486..4.336 rows=4581.00 loops=1)	
8	Storage: Memory Maximum Storage: 17kB	
9	Buffers: shared hit=3 read=25, temp read=10 written=12	
10	-> Sort (cost=412.37..423.82 rows=4581 width=6) (actual time=3.470..3.812 rows=4581.00 loops=1)	
11	Sort Key: i.film_id	
12	Sort Method: external merge Disk: 80kB	
13	Buffers: shared hit=3 read=25, temp read=10 written=12	
14	-> Seq Scan on inventory i (cost=0.00..70.81 rows=4581 width=6) (actual time=0.652..1.689 rows=4581.00 loops=1)	
15	Buffers: shared read=25	
16	Planning:	
17	Buffers: shared hit=86 read=5	
18	Planning Time: 5.536 ms	
19	Execution Time: 6.778 ms	
Total rows: 19	Query complete 00:00:00.032	L

Question:

7. Do you see a Sort node? Or did it use Index Scan to get sorted data for free?

Yes, there is a Sort node. The inventory table is sorted by i.film_id. The film table uses an Index Scan on film_pkey, which provides the required ordering without an explicit Sort.

8. Reset the settings (RESET ALL;) and remove the ORDER BY. Does it switch back to Hash Join? Why?

20	EXPLAIN (ANALYZE, BUFFERS)
21	SELECT f.film_id, f.title, i.inventory_id
22	FROM film f
23	JOIN inventory i
24	ON f.film_id = i.film_id

Data Output	Messages	Notifications
Showing rows: 1 to 16		
Page No: 1 of 1		
2	Merge Cond: (f.film_id = i.film_id)	
3	Buffers: shared hit=109	
4	-> Index Scan using film_pkey on film f (cost=0.28..127.33 rows=1000 width=19) (actual time=0.027..0.404 rows=1000.00 loops=1)	
5	Index Searches: 1	
6	Buffers: shared hit=84	
7	-> Sort (cost=349.37..360.82 rows=4581 width=6) (actual time=1.021..1.172 rows=4581.00 loops=1)	
8	Sort Key: i.film_id	
9	Sort Method: quicksort Memory: 300kB	
10	Buffers: shared hit=25	
11	-> Seq Scan on inventory i (cost=0.00..70.81 rows=4581 width=6) (actual time=0.022..0.491 rows=4581.00 loops=1)	
12	Buffers: shared hit=25	
13	Planning:	
14	Buffers: shared hit=6	
15	Planning Time: 0.560 ms	
16	Execution Time: 2.411 ms	
Total rows: 16	Query complete 00:00:00.035	LF Ln 24, Col 29

After resetting the configuration and removing the ORDER BY clause, PostgreSQL still

```
29 EXPLAIN ANALYZE
30 SELECT
31     c.first_name,
32     c.last_name,
33     r.rental_date,
34     i.inventory_id
35 FROM customer c
36 JOIN rental r
37     ON c.customer_id = r.customer_id
38 JOIN inventory i
39     ON r.inventory_id = i.inventory_id;
```

Data Output Messages Notifications

Showing rows: 1 to 22 Page No: 1 of 1

	QUERY PLAN
8	Buffers: shared hit=150
9	-> Hash (cost=14.99..14.99 rows=599 width=17) (actual time=0.147..0.149 rows=599.00 loops=1)
10	Buckets: 1024 Batches: 1 Memory Usage: 39kB
11	Buffers: shared hit=9
12	-> Seq Scan on customer c (cost=0.00..14.99 rows=599 width=17) (actual time=0.009..0.070 rows=599.00 loops=1)
13	Buffers: shared hit=9
14	-> Hash (cost=70.81..70.81 rows=4581 width=4) (actual time=1.520..1.520 rows=4581.00 loops=1)
15	Buckets: 8192 Batches: 1 Memory Usage: 226kB
16	Buffers: shared hit=25
17	-> Seq Scan on inventory i (cost=0.00..70.81 rows=4581 width=4) (actual time=0.005..0.666 rows=4581.00 loops=1)
18	Buffers: shared hit=25
19	Planning:
20	Buffers: shared hit=64
21	Planning Time: 0.652 ms
22	Execution Time: 14.676 ms

Total rows: 22 Query complete 00:00:00.035

chose a Merge Join. This shows that Merge Join can still be considered the cheapest execution strategy even when sorted output is not explicitly required. The main difference is that the Sort operation now uses quicksort in memory instead of external merge on disk. The execution time also decreased from 6.778 ms to 2.411 ms.

Hands-on Lab: Optimizing dvdrental

Task 1: Influence the Optimizer

Query Query History

```
1 RESET ALL;
2 SET enable_indexscan = off;
3
4 EXPLAIN ANALYZE
5 SELECT *
6 FROM rental
7 WHERE rental_date > '2005-05-24';
```

Showing rows: 1 to 7 of 1

Showing rows: 1 to 7 of 1

	QUERY PLAN	
	text	
1	Seq Scan on rental (cost=0.00..350.55 rows=16044 width=36) (actual time=0.017..3.019 rows=16044.00 loop...	
2	Filter: (rental_date > '2005-05-24 00:00:00'::timestamp without time zone)	
3	Buffers: shared hit=150	
4	Planning:	
5	Buffers: shared hit=84 read=4 dirtied=1	
6	Planning Time: 2.480 ms	
7	Execution Time: 3.947 ms	

9

SET enable_indexscan = on;

10

EXPLAIN ANALYZE

11

SELECT *

12

FROM rental

13

WHERE rental_date > '2005-05-24';

Data Output

Messages

Notifications

Showing rows: 1 to 7

Page No: 1 of 1

QUERY PLAN

text

1

Seq Scan on rental (cost=0.00..350.55 rows=16044 width=36) (actual time=0.014..2.876 rows=16044.00 loop...

2

Filter: (rental_date > '2005-05-24 00:00:00'::timestamp without time zone)

3

Buffers: shared hit=150

4

Planning:

5

Buffers: shared hit=3

6

Planning Time: 0.150 ms

7

Execution Time: 3.919 ms

Total rows: 7

Query complete 00:00:00.026

Setting	Plan	Execution Time
enable_indexscan = OFF	Seq Scan	3.947 ms
enable_indexscan = ON	Seq Scan	3.919 ms

Although index scans were enabled, PostgreSQL still chose a Sequential Scan because the query returned almost all rows in the rental table. Therefore, scanning the entire table was estimated to be more efficient than using an index.

Task 2: Join Strategy Analysis

Query

Query History

15

EXPLAIN ANALYZE

16

SELECT

17

c.first_name,

18

c.last_name,

19

r.rental_date,

20

i.inventory_id

21

FROM customer c

22

JOIN rental r

23

ON c.customer_id = r.customer_id

24

JOIN inventory i

25

ON r.inventory_id = i.inventory_id;

Data Output

Messages

Notifications

Showing rows: 1 to 22

Page No: 1 of 1

QUERY PLAN

text

1

Hash Join (cost=150.55..545.57 rows=16044 width=25) (actual time=3.130..13.330 rows=16044.00 loops=1)

2

Hash Cond: (r.inventory_id = i.inventory_id)

3

Buffers: shared hit=177 read=7

4

-> Hash Join (cost=22.48..375.33 rows=16044 width=25) (actual time=1.304..7.309 rows=16044.00 loops=1)

5

Hash Cond: (r.customer_id = c.customer_id)

6

Buffers: shared hit=152 read=7

7

-> Seq Scan on rental r (cost=0.00..310.44 rows=16044 width=14) (actual time=0.017..1.562 rows=16044.00 loop...

8

Buffers: shared hit=150

9

-> Hash (cost=14.99..14.99 rows=599 width=17) (actual time=1.270..1.270 rows=599.00 loops=1)

10

Buckets: 1024 Batches: 1 Memory Usage: 39kB

11

Buffers: shared hit=2 read=7

12

-> Seq Scan on customer c (cost=0.00..14.99 rows=599 width=17) (actual time=0.064..1.057 rows=599.00 loop...

13

Buffers: shared hit=2 read=7

14

-> Hash (cost=70.81..70.81 rows=4581 width=4) (actual time=1.797..1.797 rows=4581.00 loops=1)

15

Buckets: 8192 Batches: 1 Memory Usage: 226kB

16

Buffers: shared hit=25

17

-> Seq Scan on inventory i (cost=0.00..70.81 rows=4581 width=4) (actual time=0.028..0.835 rows=4581.00 loops=...

18

Buffers: shared hit=25

Total rows: 22

Query complete 00:00:00.044

15

EXPLAIN ANALYZE

16

SELECT

17

c.first_name,

18

c.last_name,

19

r.rental_date,

20

i.inventory_id

21

FROM customer c

22

JOIN rental r

23

ON c.customer_id = r.customer_id

24

JOIN inventory i

25

ON r.inventory_id = i.inventory_id;

Data Output

Messages

Notifications

≡+

📄

▼

📁

🗑

🗄

📥

📈

SQL

Showing rows: 1 to 22

🖨

Page No:

1

of 1

QUERY PLAN

text

🔒

5

Hash Cond: (r.customer_id = c.customer_id)

6

Buffers: shared hit=152 read=7

7

-> Seq Scan on rental r (cost=0.00..310.44 rows=16044 width=14) (actual time=0.017..1.562 rows=16044.00 loop...

8

Buffers: shared hit=150

9

-> Hash (cost=14.99..14.99 rows=599 width=17) (actual time=1.270..1.270 rows=599.00 loops=1)

10

Buckets: 1024 Batches: 1 Memory Usage: 39kB

11

Buffers: shared hit=2 read=7

12

-> Seq Scan on customer c (cost=0.00..14.99 rows=599 width=17) (actual time=0.064..1.057 rows=599.00 loop...

13

Buffers: shared hit=2 read=7

14

-> Hash (cost=70.81..70.81 rows=4581 width=4) (actual time=1.797..1.797 rows=4581.00 loops=1)

15

Buckets: 8192 Batches: 1 Memory Usage: 226kB

16

Buffers: shared hit=25

17

-> Seq Scan on inventory i (cost=0.00..70.81 rows=4581 width=4) (actual time=0.028..0.835 rows=4581.00 loops=...

18

Buffers: shared hit=25

19

Planning:

20

Buffers: shared hit=369 read=9

21

Planning Time: 6.342 ms

22

Execution Time: 14.164 ms

Total rows: 22

Query complete 00:00:00.044

Run ANALYZE on the tables

```

27 ANALYZE;
28
29 EXPLAIN ANALYZE
30 SELECT
31     c.first_name,
32     c.last_name,
33     r.rental_date,
34     i.inventory_id
35 FROM customer c
36 JOIN rental r
37     ON c.customer_id = r.customer_id
38 JOIN inventory i
39     ON r.inventory_id = i.inventory_id;

```

Data Output Messages Notifications

Showing rows: 1 to 22 Page No: 1 of 1

QUERY PLAN	
	text
1	Hash Join (cost=150.55..545.57 rows=16044 width=25) (actual time=1.717..13.764 rows=16044.00 loops=1)
2	Hash Cond: (r.inventory_id = i.inventory_id)
3	Buffers: shared hit=184
4	-> Hash Join (cost=22.48..375.33 rows=16044 width=25) (actual time=0.188..7.178 rows=16044.00 loops=1)
5	Hash Cond: (r.customer_id = c.customer_id)
6	Buffers: shared hit=159
7	-> Seq Scan on rental r (cost=0.00..310.44 rows=16044 width=14) (actual time=0.008..1.745 rows=16044.00 loop...)
8	Buffers: shared hit=150
9	-> Hash (cost=14.99..14.99 rows=599 width=17) (actual time=0.147..0.149 rows=599.00 loops=1)
10	Buckets: 1024 Batches: 1 Memory Usage: 39kB
11	Buffers: shared hit=9
12	-> Seq Scan on customer c (cost=0.00..14.99 rows=599 width=17) (actual time=0.009..0.070 rows=599.00 loop...)
13	Buffers: shared hit=9
14	-> Hash (cost=70.81..70.81 rows=4581 width=4) (actual time=1.520..1.520 rows=4581.00 loops=1)
15	Buckets: 8192 Batches: 1 Memory Usage: 226kB
Total rows: 22 Query complete 00:00:00.035	

PostgreSQL selected Hash Join for both join operations in the three-way join between customer, rental, and inventory. After running ANALYZE on the tables, the execution plan remained unchanged. This indicates that the updated statistics did not make another join strategy more cost-effective for this query.